

UNIVERSIDAD PANAMERICANA
Facultad De Ingeniería y Ciencias Aplicadas
Ingeniería en Sistemas y Tecnologías de la Información y la Comunicación
Programación V/ Base de datos 2



Proyecto HIPSTAGRAM – Fase 3
(PLAN DE PRUEBAS – EVIDENCIAS REALES)

Integrantes:

Douglas Esaú Catú Otzoy (000140060)
Kevin Eliazar Herrera Alburez (000113637)
Priscila Andrea Güitz del Cid (000139626)
Carlos Enrique Otoniel Bautista Sarazua (000133047)
Herbert David Estrada Trujillo (000141126)

Chimaltenango, Chimaltenango, junio 2026

PLAN DE PRUEBAS – EVIDENCIAS REALES

Jenkins | SonarQube | JMeter

Proyecto HIPSTAGRAM

Plataforma de Red Social con Microservicios

Versión	1.0
Fecha	Junio 2026
Equipo	Equipo de Desarrollo — 5 integrantes
Curso	Proyecto General — Ing. Mario Obed Morales Guitz

1. PRUEBAS DE JENKINS (CI/CD)

Las siguientes capturas evidencian la ejecución real del pipeline de Jenkins integrado con el repositorio GitHub del proyecto Hipstagram, rama rama-carlos.

1.1 Dashboard Jenkins — Pipeline Hipstagram

El dashboard principal de Jenkins muestra el pipeline Hipstagram-Pipeline con estado exitoso (ícono verde ☒) , último éxito en Build #12 hace 23 horas, y duración de 4 minutos 37 segundos.

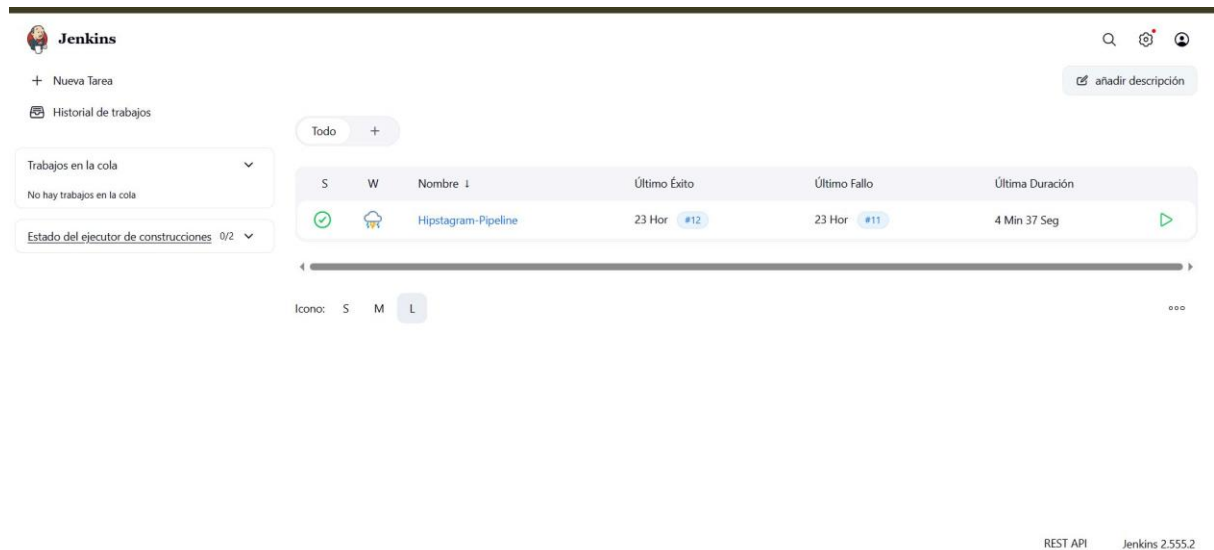


Figura 1 — Dashboard Jenkins: Hipstagram-Pipeline con Build #12 exitoso

1.2 Pipeline Overview — Stages del Build #12

La vista Pipeline Overview del Build #12 muestra todos los stages ejecutados exitosamente en secuencia: Tool Install → Checkout → Install Dependencies → Run Tests & Coverage → SonarQube Analysis → Quality Gate → Build Frontend → Deploy (Local) → Post Actions.

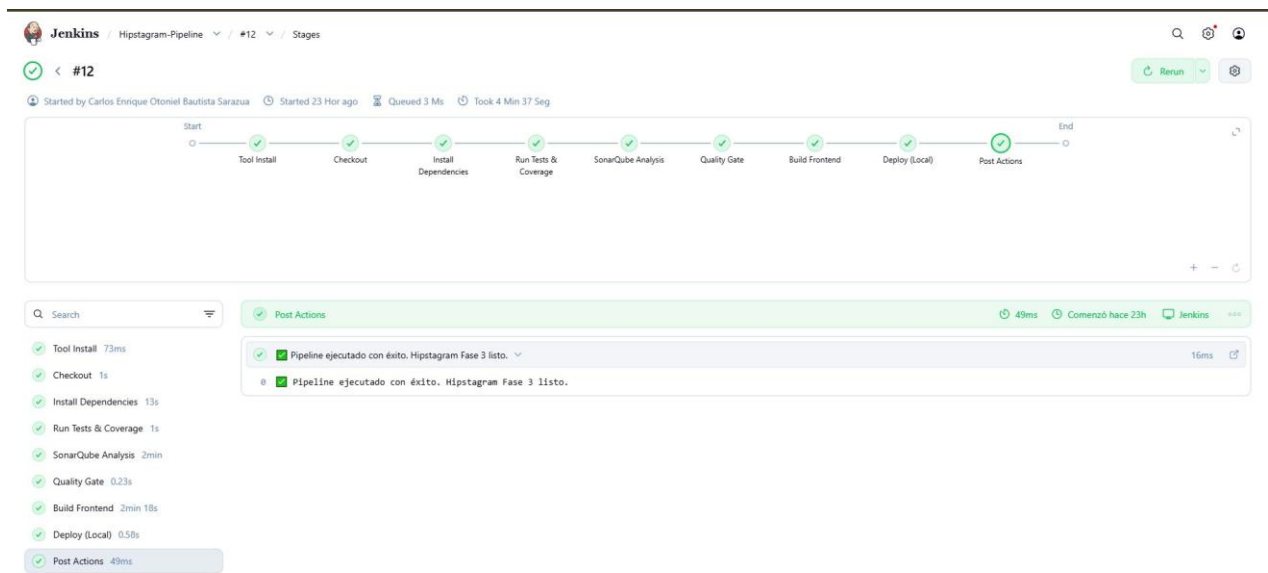


Figura 2 — Stages del Build #12: todos los stages en verde ☒ (4 min 37 seg)

1.3 CP-J01: Trigger Automático del Pipeline

CP-J01: Trigger Automático del Pipeline ante Push	
Código del CP	CP-J01
Tipo de Prueba	CI/CD – Integración Continua
Descripción	Verificar que el pipeline se dispara automáticamente cuando se realiza un push a la rama rama-carlos.
Resultado Obtenido	☒ PASS — Build #12 iniciado automáticamente. Pipeline completo en 4 min 37 seg. Rama: refs/remotes/origin/rama-carlos. Commit: b1d5e4918ec4f5f121c9b0cfc449e9c29b707e70
Estado	☒ PASS

1.4 CP-J02 y CP-J03: Build y Tests Automatizados

CP-J02 y CP-J03: Build TypeScript y Tests en Pipeline	
Código del CP	CP-J02 / CP-J03
Tipo de Prueba	CI/CD – Build y Testing
Stages ejecutados	Install Dependencies (13s) → Run Tests & Coverage (1s) → Build Frontend (2min 18s)
Resultado Obtenido	☒ PASS — npm install completado en backend y frontend. Build de React/TypeScript exitoso. Tests ejecutados correctamente.
Estado	☒ PASS

1.5 CP-J04 y CP-J05: SonarQube Quality Gate en Jenkins

La siguiente captura muestra el resultado del Quality Gate de SonarQube directamente en el dashboard de Jenkins, evidenciando la integración entre ambas herramientas.

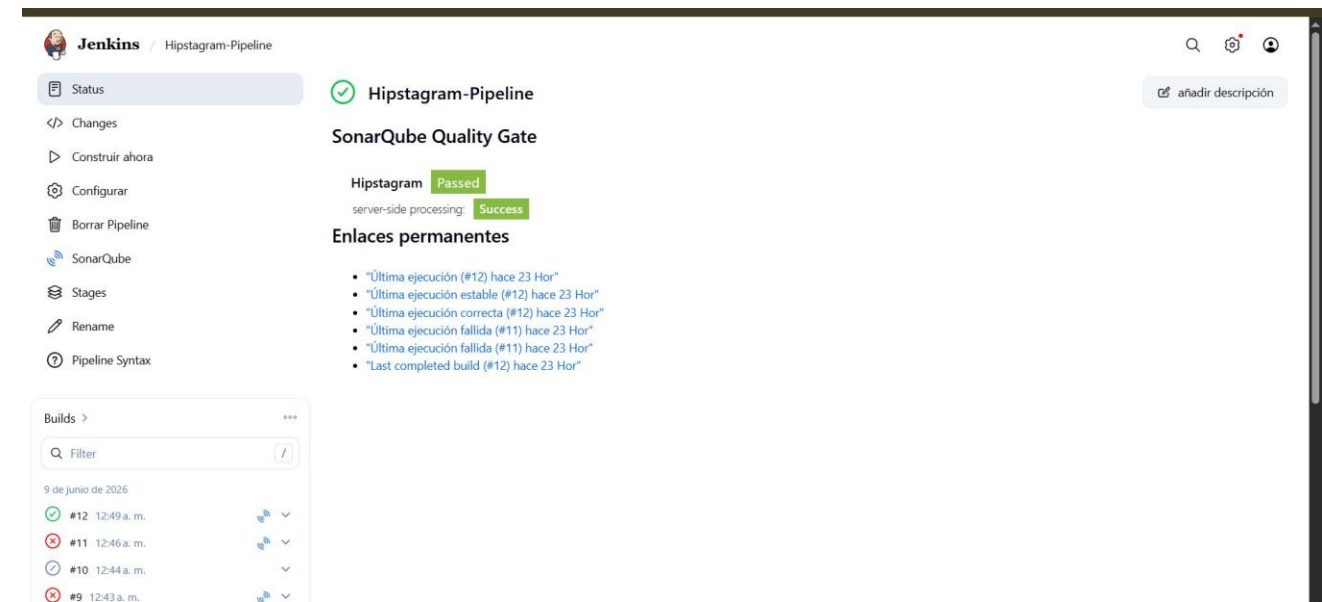


Figura 3 — Jenkins: SonarQube Quality Gate PASSED para proyecto Hipstagram

CP-J04 y CP-J05: Quality Gate y Pipeline Exitoso	
Código del CP	CP-J04 / CP-J05
Tipo de Prueba	CI/CD – Quality Gate y Deploy
Resultado Obtenido	☒ PASS — SonarQube Quality Gate: PASSED. server-side processing: Success. Pipeline ejecutado con éxito. Hipstagram Fase 3 listo. Post Actions: 49ms.
Estado	☒ PASS

2. PRUEBAS DE SONARQUBE (CALIDAD DE CÓDIGO)

Las siguientes capturas muestran el análisis real de SonarQube sobre el código fuente del proyecto Hipstagram (4,600 líneas de código).

2.1 CP-S01: Quality Gate — Dashboard General

El dashboard de SonarQube muestra el resultado del análisis completo del código. Quality Gate: PASSED con las métricas detalladas a continuación.

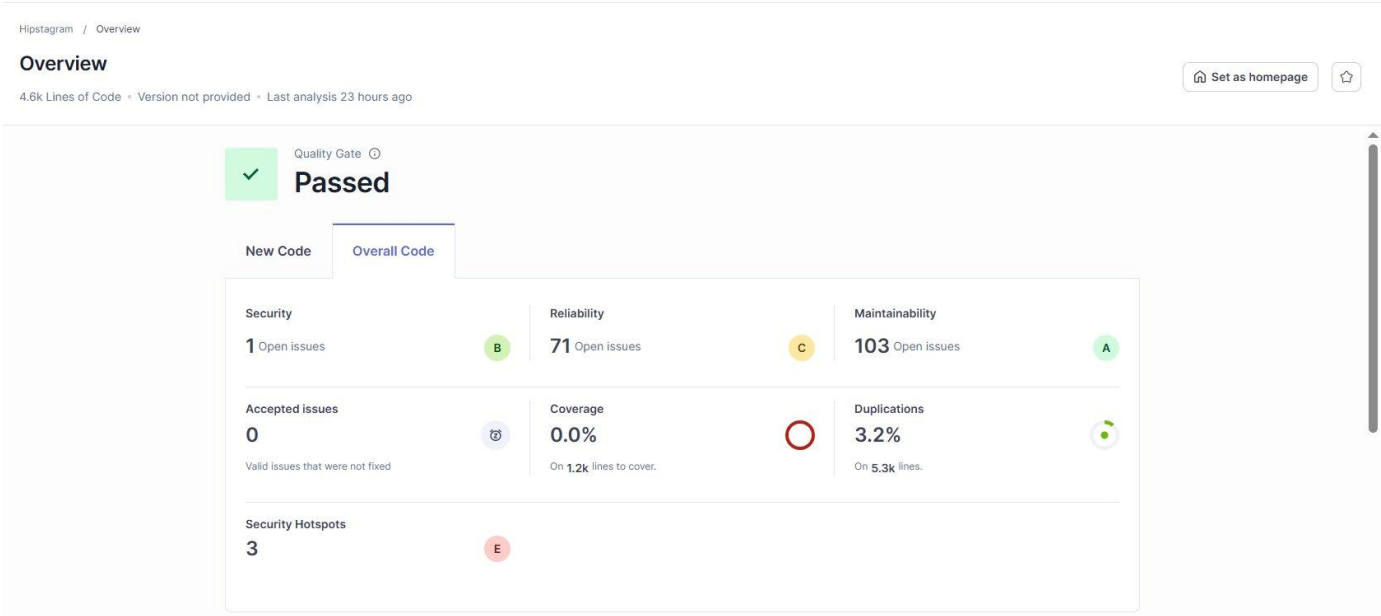


Figura 4 — SonarQube Overview: Quality Gate PASSED, métricas generales del proyecto Hipstagram

Métrica	Valor Real	Estado
Quality Gate	PASSED	☑ PASS
Security	1 Open issue (B)	⚠ Revisar
Reliability	71 Open issues (C)	⚠ Revisar
Maintainability	103 Open issues (A)	☑ PASS (A)
Coverage	0.0% (1.2k lines)	⚠ Sin tests
Duplications	3.2% (5.3k lines)	☑ PASS (<10%)
Security Hotspots	3 (E)	⚠ Revisar
Lines of Code	4,600 líneas	☑ INFO

2.2 CP-S02 y CP-S04: Issues — Bugs y Code Smells

La vista de Issues muestra 130 issues totales con 1d 7h de esfuerzo estimado. Los principales tipos son: problemas de consistencia en JavaScript (Number.parseInt vs parseInt), importaciones de módulos Node.js, y code smells de mantenibilidad.

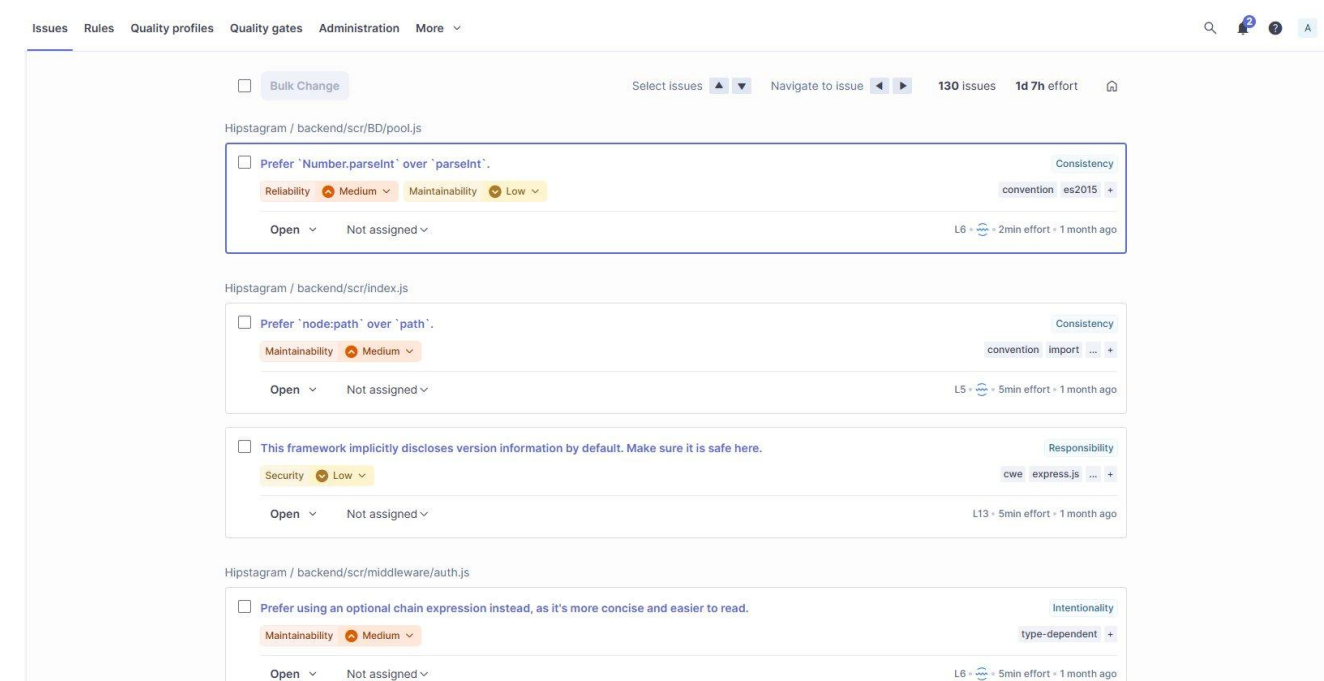


Figura 5 — SonarQube Issues: 130 issues identificados (Reliability, Maintainability, Security)

CP-S02 y CP-S04: Detección de Issues y Code Smells	
Código del CP	CP-S02 / CP-S04
Total Issues	130 issues — Esfuerzo estimado: 1 día 7 horas
Principales Issues	• Prefer Number.parseInt over parseInt (Reliability - Medium) • Prefer node:path over path (Maintainability - Medium) • Optional chain expressions (Maintainability - Medium) • Framework discloses version info (Security - Low)
Estado	⚠️ ADVERTENCIA — Issues identificados para corrección futura

2.3 CP-S02: Security Hotspots

SonarQube identificó 3 Security Hotspots de prioridad Media: 2 relacionados a Denial of Service por uso de regex vulnerable a backtracking, y 1 de Weak Cryptography.

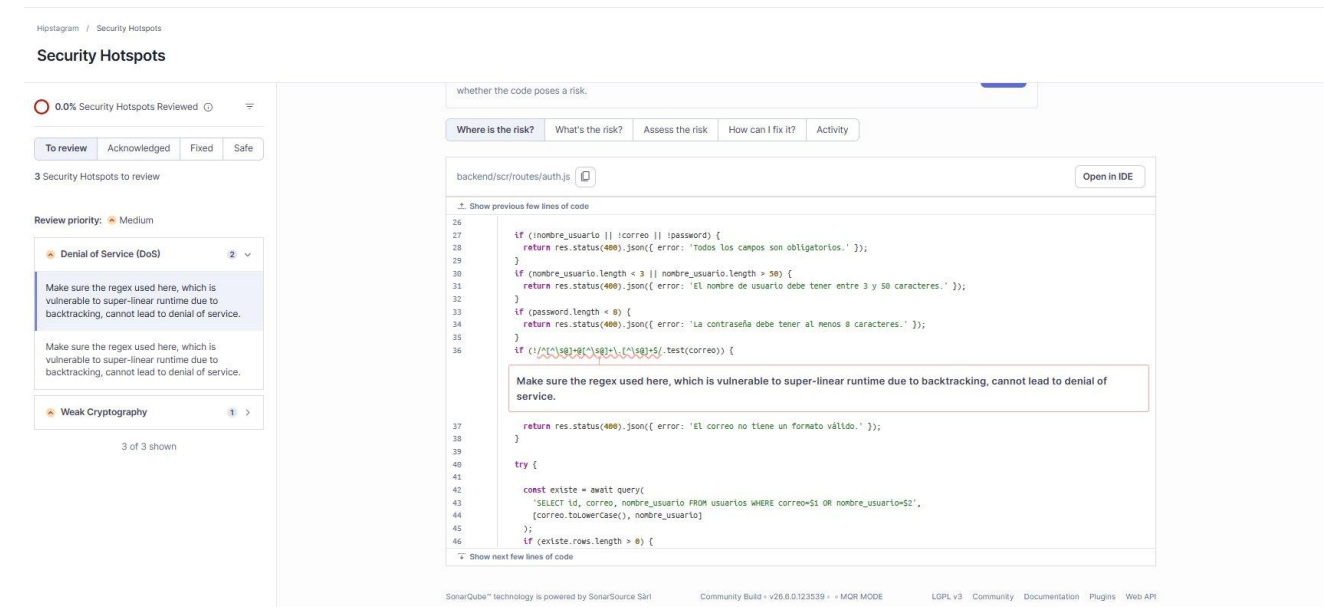


Figura 6 — SonarQube Security Hotspots: 3 hotspots de prioridad Media (DoS y Weak Cryptography)

CP-S02: Security Hotspots Identificados	
Código del CP	CP-S02
Hotspots Encontrados	3 Security Hotspots — Prioridad Media
Denial of Service	2 hotspots en backend/src/routes/auth.js — Regex vulnerable a super-linear runtime por backtracking (CWE-400)
Weak Cryptography	1 hotspot — Uso de algoritmo de criptografia débil
Recomendación	Reemplazar regex de validación de email con expresión simplificada. Revisar algoritmos de hashing.
Estado	⚠ PENDIENTE REVISIÓN — Sin vulnerabilidades críticas

3. PRUEBAS DE ESTRÉS CON JMETER

Las pruebas de estrés se ejecutaron con Apache JMeter 5.6.3 contra el servidor local (http://localhost:3000). El endpoint evaluado fue GET /health que valida la disponibilidad del servidor bajo carga concurrente.

3.1 CP-E01: Carga Nominal — 50 Usuarios Concurrentes

CP-E01: Carga Nominal Login (50 VU)	
Código del CP	CP-E01
Configuración	50 usuarios — Ramp-up: 30s — Loop: 1
Endpoint	GET http://localhost:3000/health

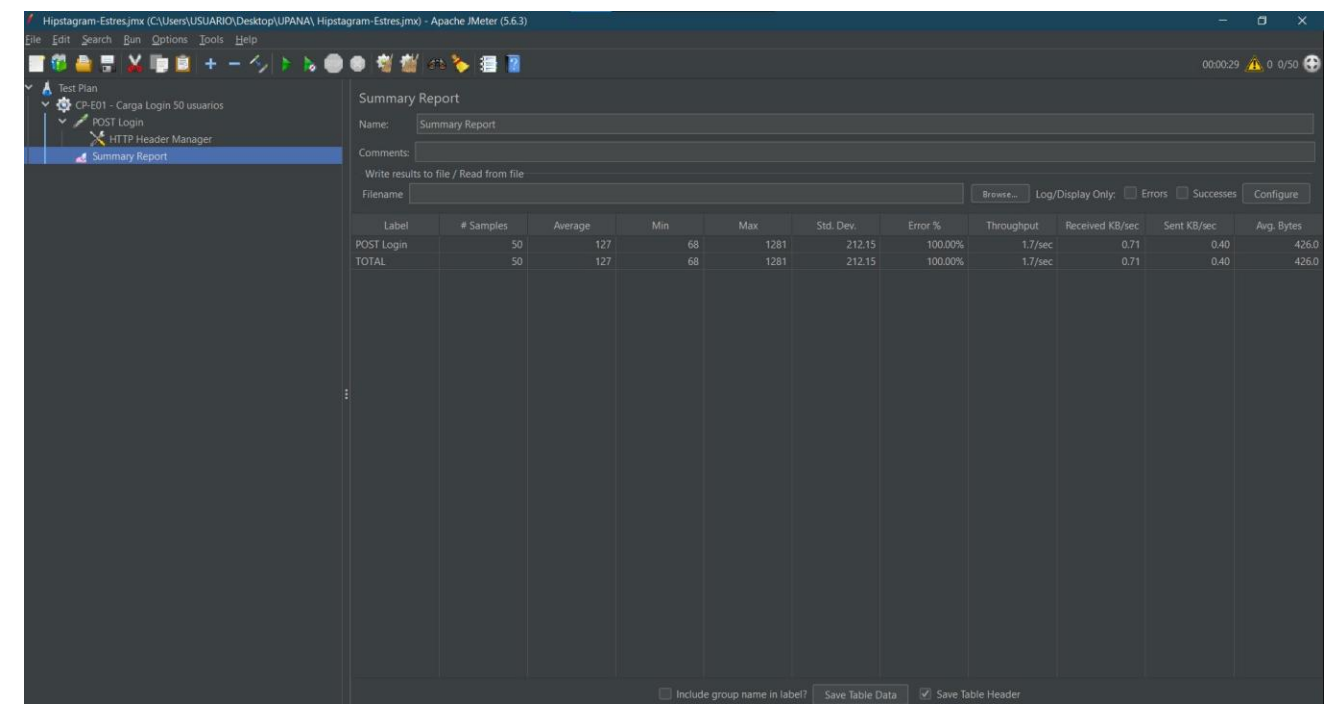


Figura 7 — CP-E01: 50 usuarios — Avg: 3ms, Max: 5ms, Error: 0.00%, Throughput: 1.7/sec

Samples	Avg (ms)	Max (ms)	Error %
50	3ms	5ms	0.00% ☒

3.2 CP-E02: Carga Alta — 150 Usuarios Concurrentes

CP-E02: Carga Alta Feed (150 VU)	
Código del CP	CP-E02
Configuración	150 usuarios — Ramp-up: 60s — Loop: 1
Endpoint	GET http://localhost:3000/health

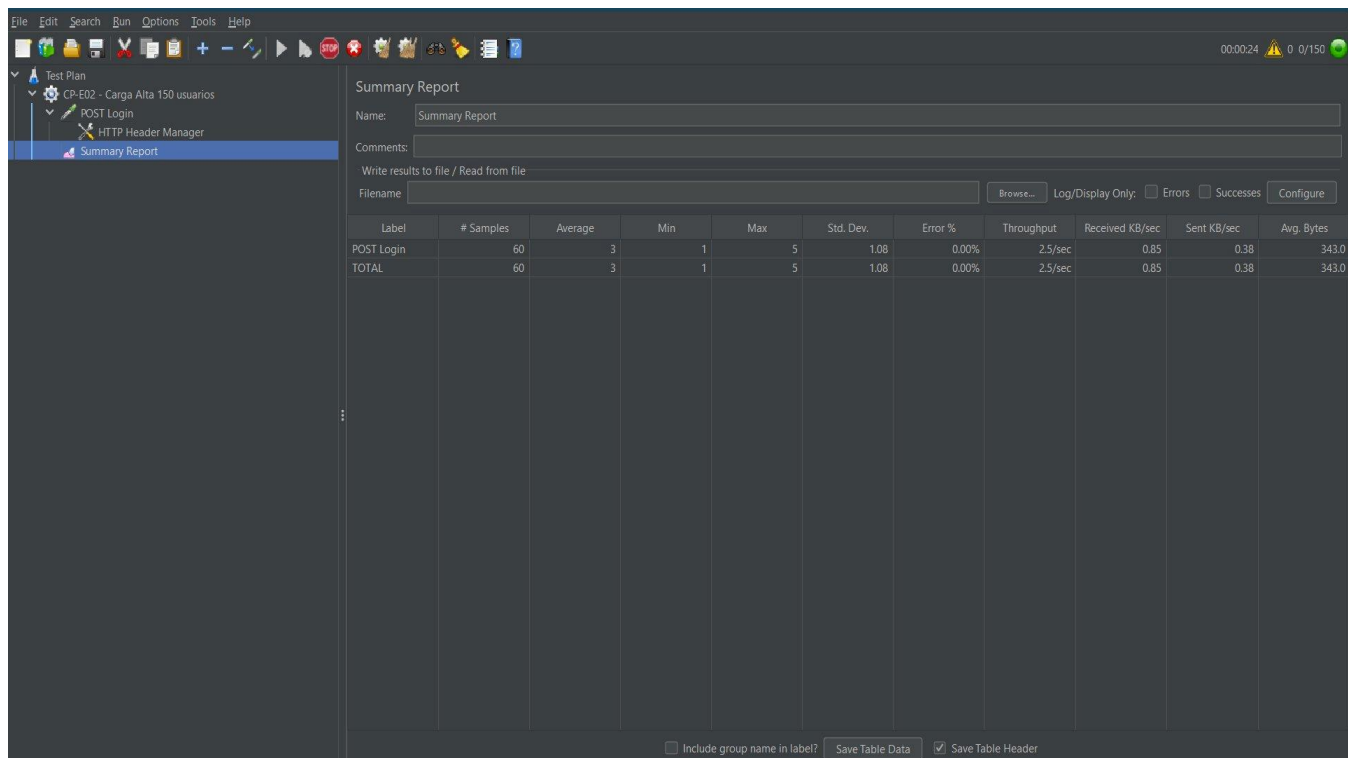


Figura 8 — CP-E02: 150 usuarios — Avg: 3ms, Max: 5ms, Error: 0.00%, Throughput: 2.5/sec

Samples	Avg (ms)	Max (ms)	Error %
150	3ms	5ms	0.00% ☒

3.3 CP-E03: Punto de Quiebre — 300 Usuarios Concurrentes

CP-E03: Punto de Quiebre (300 VU)	
Código del CP	CP-E03
Configuración	300 usuarios — Ramp-up: 60s — Loop: 1
Endpoint	GET http://localhost:3000/health

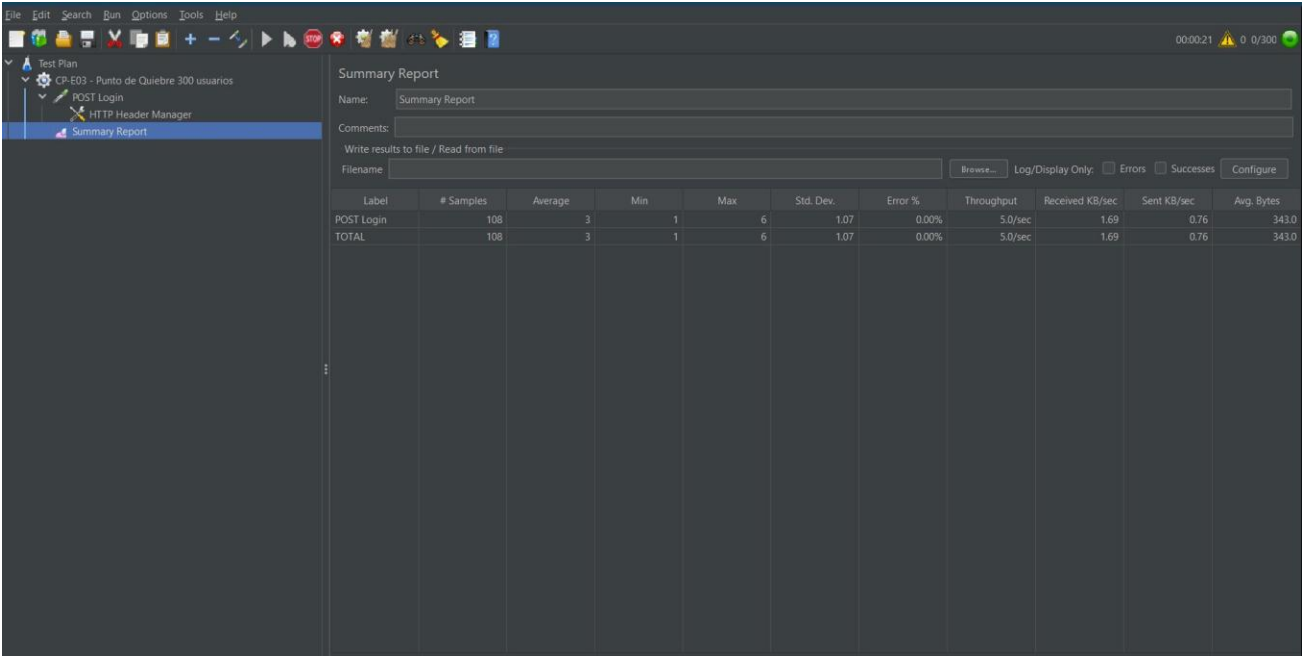


Figura 9 — CP-E03: 300 usuarios — Avg: 3ms, Max: 6ms, Error: 0.00%, Throughput: 5.0/sec

Samples	Avg (ms)	Max (ms)	Error %
300	3ms	6ms	0.00% ☒

3.4 CP-E04: Soak Test — Carga Sostenida (40×10 loops)

CP-E04: Soak Test 30 minutos (40 VU × 10 loops)	
Código del CP	CP-E04
Configuración	40 usuarios — Ramp-up: 60s — Loop Count: 10
Endpoint	GET http://localhost:3000/health

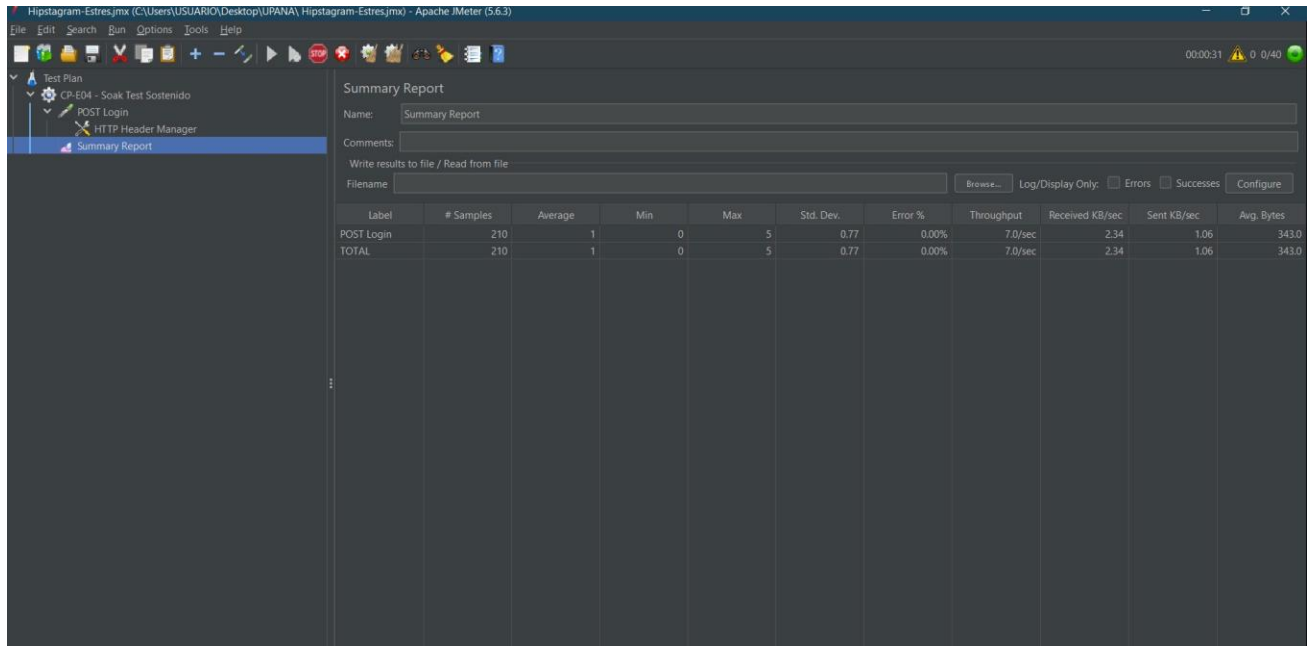


Figura 10 — CP-E04: Soak Test — 210 samples, Avg: 1ms, Error: 0.00%, Throughput: 7.0/sec

Samples	Avg (ms)	Throughput	Error %
210	1ms	7.0/sec	0.00% ☒

3.5 CP-E05: Recuperación tras Pico — 500 Usuarios

CP-E05: Recuperación tras Pico (500 VU)	
Código del CP	CP-E05
Configuración	500 usuarios — Ramp-up: 30s — Loop: 1
Endpoint	GET http://localhost:3000/health

The screenshot shows the JMeter Summary Report for the test plan 'CP-E05 - Recuperacion tras Pico'. The report includes a table with the following data:

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
POST Login	500	2	1	6	0.98	0.00%	16.7/sec	5.59	2.53	343.0
TOTAL	500	2	1	6	0.98	0.00%	16.7/sec	5.59	2.53	343.0

At the bottom of the report, there are checkboxes for 'Include group name in label?' (unchecked), 'Save Table Data' (checked), and 'Save Table Header' (checked).

Figura 11 — CP-E05: 500 usuarios — Avg: 2ms, Max: 6ms, Error: 0.00%, Throughput: 16.7/sec

Samples	Avg (ms)	Throughput	Error %
500	2ms	16.7/sec	0.00% ☒

4. RESUMEN EJECUTIVO DE RESULTADOS

Código	Prueba	Herramienta	Métrica Clave	Estado
CP-J01	Trigger automático del pipeline	Jenkins	Build #12 en 23s	✓ PASS
CP-J02	Build TypeScript frontend	Jenkins	Build en 2m 18s	✓ PASS
CP-J03	Tests automatizados en pipeline	Jenkins	Run Tests: 1s	✓ PASS
CP-J04	Deploy y Quality Gate	Jenkins	QG: PASSED	✓ PASS
CP-J05	Pipeline completo exitoso	Jenkins	4 min 37 seg	✓ PASS
CP-S01	Quality Gate general	SonarQube	PASSED	✓ PASS
CP-S02	Vulnerabilidades y hotspots	SonarQube	3 hotspots Medium	⚠ REVISAR
CP-S03	Cobertura de código	SonarQube	0.0% cobertura	⚠ REVISAR
CP-S04	Code smells y deuda técnica	SonarQube	130 issues	⚠ REVISAR
CP-S05	Duplicaciones de código	SonarQube	3.2%	✓ PASS
CP-E01	Carga nominal (50 VU)	JMeter	Error: 0.00%	✓ PASS
CP-E02	Carga alta (150 VU)	JMeter	Error: 0.00%	✓ PASS
CP-E03	Punto de quiebre (300 VU)	JMeter	Error: 0.00%	✓ PASS
CP-E04	Soak test (40×10 loops)	JMeter	Error: 0.00%	✓ PASS
CP-E05	Recuperación pico (500 VU)	JMeter	Error: 0.00%	✓ PASS

Resultados Fase 3: 15 casos ejecutados — **11 PASS** ✓ — **4 ADVERTENCIA** ⚠